MARIOH: Multiplicity-Aware Hypergraph Reconstruction (Supplementary Document)

A. Additional Case Studies

In Figures 1 and 2, we present sub-hypergraph reconstruction results on two distinct datasets (Crime and Host-Virus). Each dataset was randomly split in half, with one portion (50%) used for training and the other (50%) for testing, following the same setup as in the main paper. We then extracted sub-hypergraphs from the test portion of each dataset and reconstructed them using MARIOH and SHyRe-Count, both trained on their respective training subsets.

For the Crime dataset, we focus on an ego-network centered on *Cheatham Barry*. In this dataset, each hyperedge represents a set of individuals involved in the same criminal incident. MARIOH exactly recovers all hyperedges (Jaccard similarity = 1.000), whereas SHyRe-Count recovers only a subset with some false positives (Jaccard similarity = 0.800).

For the Host-Virus dataset, we extract an ego-network centered on *Macaca nemestrina*. In this dataset, each hyperedge corresponds to a set of hosts infected by the same virus. MARIOH exactly recovers all hyperedges (Jaccard and multi-Jaccard similarity = 1.000), whereas SHyRe-Count suffers from significant false positives (Jaccard similarity = 0.222).

B. Negative Sampling

We generate hyperedge candidates for classifier training. Positive samples consist of true hyperedges taken directly from $\mathcal{H}^{(S)}$. Negative samples, on the other hand, are cliques of the same size as the true hyperedges but are not present in $\mathcal{H}^{(S)}$. These negative samples are obtained by sampling subcliques from larger cliques. Specifically, maximal cliques in the projected graph after filtering $\mathcal{G}'$ (as detailed in Sect. III-B) are grouped by size. For each true hyperedge size, we attempt to generate negative samples by randomly selecting subcliques of the same size from cliques larger than the true hyperedge size. If no larger cliques are available, we may not generate negative samples of that size, resulting in fewer negative samples than desired. We used a negative sampling ratio of $1:4$, meaning that for each positive sample, we generated at most four negative samples.

Lemma 1 (Time Complexity of Negative Sampling)**.** *Given graph $\mathcal{G}$, and N_p ground truth hyperedges with maximum size $s_{\max}$, the negative sampling algorithm generates $O(N_p)$ negative samples in total time*

$$O\left(\mathcal{C}_{total} + N_p \cdot s_{\max}\right),$$

where $\mathcal{C}_{total}$ is the total number of maximal cliques in graph $\mathcal{G}$, and $s_{\max}$ is the maximum hyperedge size.

Proof. The negative sampling algorithm begins by finding all maximal cliques in the graph $\mathcal{G}$, which takes $O(C_{\text{total}})$ time [2], where C_{total} denotes the total number of maximal cliques. These cliques are then organized by their sizes using a hash map, requiring an additional $O(C_{\text{total}})$ time. For each ground truth hyperedge h of size s, the algorithm attempts to generate up to K (spec., 4) negative samples of the same size. It collects all maximal cliques of size at least s; if no such cliques exist, it cannot generate negative samples for hyperedges of size s and proceeds to the next hyperedge.

When larger cliques are available, the algorithm makes up to $K \times 5$ attempts to generate negative samples for each h. In each attempt, it randomly selects a larger clique C and samples s nodes to form a candidate hyperedge h'. It then checks whether h' is neither in the ground truth set $\mathcal{H}^{(S)}$ nor already sampled; this check takes $O(1)$ time using efficient data structures like hash sets. If h' is valid, it is added to the set of negative samples. The attempt counter ensures that the algorithm does not run indefinitely when valid negative samples are scarce, particularly if many sampled sub-cliques are already in the ground truth or previously sampled.

Since K are constants, the time spent per hyperedge h is $O(s)$, where $s \leq s_{\max}$. Summing over all N_p ground truth hyperedges, the total time for negative sample generation is $O(N_p \cdot s_{\max})$. Therefore, the overall time complexity of the negative sampling algorithm is $O(C_{\text{total}} + N_p \cdot s_{\max})$. □

C. Multiplicity-Aware Classifier

We use a 2-layered Multi-Layer Perceptron (MLP) to predict whether a given clique is a true hyperedge. The input to the classifier is the multiplicity-aware feature vector. The output layer uses a softmax activation to produce a probability distribution over two classes. The classifier is trained to minimize the binary cross-entropy loss using the predicted and ground-truth labels (1 for true hyperedges, 0 for non-hyperedges). Hidden layer dimensions and dropout rates are optimized depending on the datasets. A detailed hyper-parameter search space is described below.

Hyperparameter Search: We optimized the hyperparameters of the classifier using Bayesian optimization with the Optuna library [3]. Negative sampling was employed to generate the validation dataset from $\mathcal{G}^{(S)}$, and 5-fold cross-validation was performed to identify the hyperparameters that maximize AUROC. The search space included:

- **Batch Size**: {256, 512, 1024, 2048, 4096}
- **Number of Epochs**: $[1, 50]$

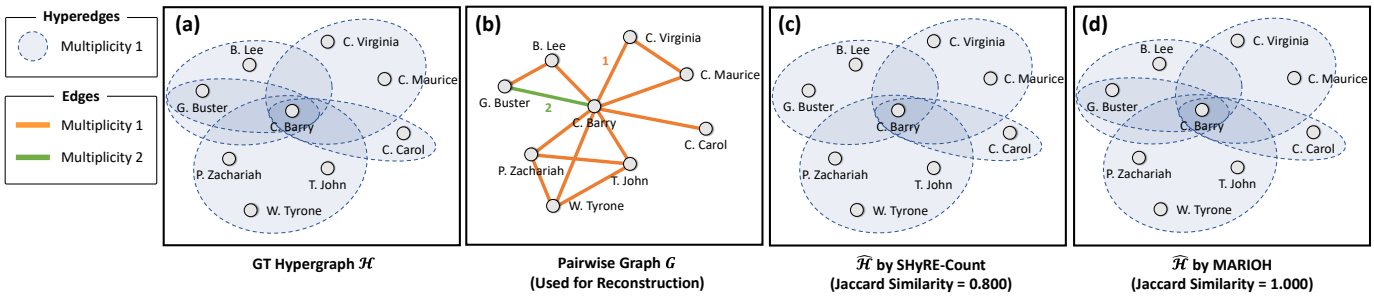


Fig. 1: Example of hypergraph reconstruction on a Crime dataset. **(a):** In the ground-truth hypergraph, each hyperedge represents a set of people involved in the same criminal incident. We focus on an ego network centered on *Cheatham Barry*. **(b):** As input, the graph representation G of $\mathcal{H}$ is given, where each edge indicates the frequency of co-occurrences in criminal incidents. **(c):** SHyRE-Count [1] recovers the original hyperedges in $\mathcal{H}$ with some false positives (Jaccard similarity $=$ 0.800). **(d):** In contrast, our proposed method, MARIOH, exactly restores $\mathcal{H}$ (Jaccard similarity $=$ 1.000).

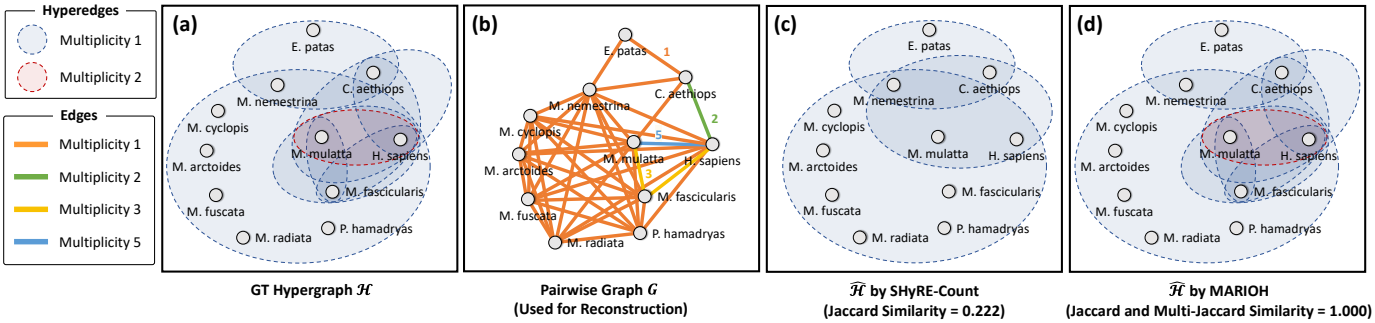


Fig. 2: Example of hypergraph reconstruction on a Host-Virus dataset. **(a):** In the ground-truth hypergraph, each hyperedge represents a set of hosts infected by the same virus. We focus on an ego network centered on *Macaca nemestrina* (sampling up to 10 neighbors). **(b):** As input, the graph representation G of $\mathcal{H}$ is given, where each edge indicates the number of co-infections between hosts. **(c):** SHyRE-Count [1] recovers the original hyperedges in $\mathcal{H}$ with some false positives (Jaccard similarity $=$ 0.222). **(d):** In contrast, our proposed method, MARIOH, exactly restores $\mathcal{H}$ (Jaccard and multi-Jaccard similarity $=$ 1.000).

- **Learning Rate**: $[10^{-5}, 10^{-2}]$
- **Hidden Layer Dimensions**: {(128, 64), (128, 128), (256, 128), (256, 256)}
- **Dropout Rate**: $[0.1, 0.5]$

After 100 trials, the best hyperparameters achieving the highest mean AUROC during cross-validation were selected.

D. Baseline Methods for Hypergraph Reconstruction

- **Overlapping Community Detection**
  - **CFinder [4]** identifies communities by finding overlapping k-cliques. The optimal k is selected using a Jaccard-based metric on the training set.
  - **Demon [5]** identifies overlapping communities in a graph by iteratively aggregating local communities discovered around each node. It uses a threshold parameter ϵ to merge similar communities and a minimum community size to filter out small communities.
- **Clique Decomposition**
  - **Clique Covering [6]** covers all edges in a graph by identifying cliques that include uncovered edges. It iteratively selects an uncovered edge, expands it into a maximal clique using neighboring nodes, and removes edges covered by the clique until all edges are included in at least one clique.
  - **Max Clique [2]** considers all maximal cliques in the graph as hyperedges.
- **Hypergraph Reconstruction**
  - **Bayesian-MDL [7]** reconstructs hypergraphs using a Bayesian generative model, emphasizing the principle of parsimony to infer the simplest hypergraph that explains the observed pairwise data. It calculates the posterior probability of a hypergraph and estimates structures through Markov Chain Monte Carlo sampling.
  - **SHyRE [1]** reconstructs hypergraphs by sampling cliques from the projected graph based on a statistical distribution $\rho(n, k)$, which captures patterns of hyperedges within maximal cliques. A classifier is trained to distinguish between hyperedges and non-hyperedges. SHyRE-Count uses basic structural features, while SHyRE-Motif incorporates subgraph patterns such as triangle and square motifs as additional features for classification.
  - **SHyRE-Unsup [1]** reconstructs hypergraphs by iteratively selecting maximal cliques as candidates for hyperedges. The cliques are ranked based on their size and average edge multiplicity, preferring larger cliques with lower average edge multiplicities. Once selected, a clique is converted into a hyperedge, and the corresponding edge multiplicities in the projected graph are reduced until all edge multiplicities are satisfied.

TABLE I: Execution time comparison. Average runtime (in seconds) across datasets, reported with standard deviations over five runs. "OOT" indicates "out of time" (exceeding 24 hours), and "OOM" represents "out of memory" (system limit: 384 GB).

Dataset	CFinder	Demon	Clique Covering	Max Clique	Bayesian-MDL	SHyRe-Unsup	SHyRe-Motif	SHyRe-Count	MARIOH
Enron	0.23 ± 0.00	0.15 ± 0.00	0.01 ± 0.00	0.03 ± 0.00	0.46 ± 0.00	1.55 ± 0.29	22.89 ± 0.69	1.79 ± 0.38	5.24 ± 0.43
P.School	277.44 ± 2.86	0.83 ± 0.00	0.06 ± 0.00	0.67 ± 0.02	9.19 ± 0.09	50.63 ± 0.44	OOT	14.59 ± 0.67	27.70 ± 0.78
H.School	3.20 ± 0.02	0.48 ± 0.01	0.03 ± 0.00	0.13 ± 0.00	1.74 ± 0.01	10.49 ± 0.13	1281.12 ± 23.79	4.40 ± 0.49	5.90 ± 0.11
Crime	0.02 ± 0.00	0.15 ± 0.02	0.01 ± 0.00	0.01 ± 0.00	0.39 ± 0.00	0.42 ± 0.00	0.27 ± 0.04	0.17 ± 0.03	0.85 ± 0.02
Hosts	0.56 ± 0.00	2.30 ± 0.02	0.01 ± 0.00	0.06 ± 0.00	2.45 ± 0.01	3.91 ± 0.01	336.64 ± 5.33	1.57 ± 0.25	4.20 ± 0.09
Directors	0.02 ± 0.00	0.12 ± 0.00	0.01 ± 0.00	0.01 ± 0.00	0.38 ± 0.00	0.10 ± 0.02	0.11 ± 0.01	0.09 ± 0.02	0.76 ± 0.03
Foursquare	8.56 ± 0.10	9.70 ± 0.11	0.16 ± 0.00	0.65 ± 0.00	28.79 ± 0.60	22.87 ± 0.17	8422.65 ± 259.82	3.75 ± 0.31	8.37 ± 0.25
DBLP	29.18 ± 0.57	18143.66 ± 579.85	8.46 ± 0.18	11.99 ± 0.67	245.16 ± 0.61	OOT	3046.27 ± 104.93	2798.34 ± 41.49	660.13 ± 14.15
Eu	16363.26 ± 2186.62	5.34 ± 1.34	0.29 ± 0.04	12.20 ± 6.15	36.77 ± 13.47	1925.37 ± 200.26	OOM	249.92 ± 2.31	1547.37 ± 196.01
TopCS	4.33 ± 0.07	270.03 ± 1.54	1.17 ± 0.01	1.54 ± 0.02	42.01 ± 0.33	5889.73 ± 87.56	1068.01 ± 14.62	103.41 ± 2.45	81.65 ± 3.95

E. Bidirectional Search Parameters

The initial classification threshold θ_{init} and the negative prediction processing ratio r were determined as the combination that maximized reconstruction accuracy on the source projected graph $G^{(S)}$ and its corresponding source hypergraph $H^{(S)}$. Specifically, hyperparameters are explored from the ranges: $\theta_t = [0.5, 0.6, \ldots, 1.0]$, $r = [5, 10, \ldots, 100]$. For all experiments, except those conducted for the parameter sensitivity analysis (Fig. 3 in the main paper), MARIOH was configured with $\alpha = 1/20$.

F. Structural Properties of Hypergraphs

We analyzed the structural characteristics of hypergraphs by quantifying and comparing the following metrics across the ground truth hypergraph ($\mathcal{H}^{(T)}$) and the reconstructed hypergraph ($\widehat{\mathcal{H}}^{(T)}$):

- **Number of Nodes and Hyperedges**: The total count of unique nodes and hyperedges in the hypergraph was computed to capture its overall size and complexity. This metric provides fundamental insights into the scale and sparsity of the hypergraph.
- **Mean Hyperedge Sizes**: We calculated the mean (μ) of the hyperedge sizes to measure their central tendency and variability. These metrics reflect the heterogeneity in the hypergraph's structural composition.
- **Degree Distributions (Single, Pairwise, Triple)**: The degree distributions of nodes, node pairs, and node triples within hyperedges were analyzed to characterize their occurrence frequencies. This metric highlights how nodes participate in single hyperedges, pairwise connections, and higher-order interactions [8]–[10].
- **Density and Overlapness**: Hypergraph density was defined as the ratio of hyperedges to nodes [11], while overlapness was measured as the ratio of the total hyperedge size to the number of nodes [10]. These metrics provide a macroscopic view of the connectivity and overlap of hyperedges within the hypergraph.
- **Simplicial Closure Ratio**: This metric measures the proportion of closed triangles formed by three mutually connected nodes within the hypergraph. A higher closure ratio indicates a greater tendency for nodes to form tightly connected substructures, which is an essential characteristic of higher-order networks [12].
- **Hyperedge Homogeneity**: To evaluate the internal consistency of hyperedges, we quantified the average pairwise connection frequency within each hyperedge [10]. This metric captures the uniformity of relationships among hyperedge members.
- **Singular Value Distribution**: We computed the singular values of the adjacency matrix derived from the projected graph representation of the hypergraph. The distribution of singular values provides insights into the structural connectivity and dimensionality of the graph [9], [13].

G. Runtime Analysis on Individual Datasets

The execution times presented in Table I highlight significant differences in runtime efficiency among the methods across various datasets. MARIOH consistently demonstrates competitive performance, with moderate execution times across all datasets. SHyRe-Count generally exhibits slightly faster runtimes than MARIOH, while SHyRe-Motif and SHyRe-Unsup show considerably higher runtimes, often reaching Out-of-Time (OOT) or Out-of-Memory (OOM) limits for larger datasets like DBLP and EU. Bayesian-MDL achieves the fastest execution times among the hypergraph reconstruction methods, followed closely by MARIOH.

Clique decomposition methods (Max Clique and Clique Covering) are the fastest overall but at the cost of significantly lower reconstruction accuracy (as shown in Table II). Community-based methods like Demon and CFinder exhibit much higher runtimes for large datasets, with CFinder being particularly slow on datasets such as EU and TopCS. These results highlight MARIOH's balance between runtime efficiency and reconstruction accuracy, making it a reliable option for hypergraph analysis.

H. Storage Savings through Hypergraph Reconstruction

To evaluate storage efficiency, we compare the memory usage of the original hypergraph, its projected graph, and the hypergraph reconstructed by MARIOH. All are represented uniformly as nested arrays, assuming that nodes in edges and hyperedges are indexed as int64. The relative storage ratio is calculated using the projected graph as the baseline (100%), with the storage requirements of the original and reconstructed hypergraphs expressed as percentages of the projected graph's memory usage.

As illustrated in Fig. 3, the reconstructed hypergraph achieves compression ratios close to the original hypergraph across various datasets, with both consistently exhibiting lower storage usage than the projected graph. This highlights the compactness of the reconstructed hypergraph and its ability to preserve higher-order interactions.

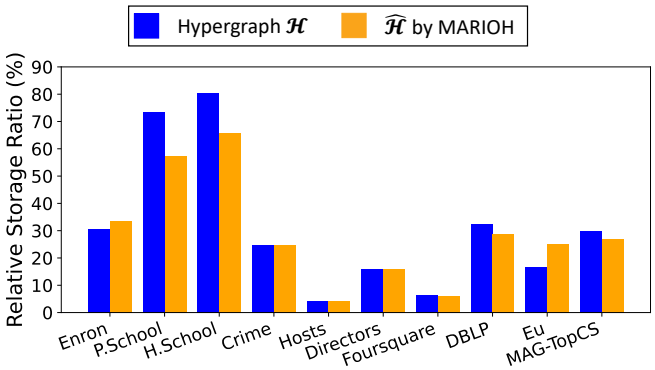
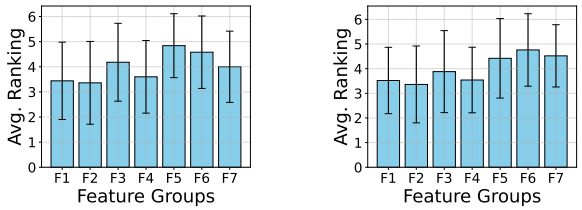


Fig. 3: Storage Efficiency: Hypergraph vs. Projected Graph. The relative storage ratio is calculated with the projected graph as the baseline (100%). Both the original and reconstructed hypergraphs consistently show lower storage usage than the projected graph across all datasets, demonstrating the compactness of the reconstructed hypergraph.



(a) Multiplicity-reduced hypergraphs

(b) Multiplicity-preserved hypergraphs

Fig. 4: Feature importance analysis. In both multiplicity-reduced and multiplicity-preserved settings, the feature F2 (Edge Multiplicity) is the most informative for determining whether a clique forms a hyperedge, among the seven groups of clique features.

I. Feature Importance Analysis

We analyze the impact of each feature group in MARIOH's multiplicity-aware classifier by evaluating reconstruction accuracy after excluding individual feature groups. The seven feature groups tested are: (F1) Clique Size (i.e., $|\mathcal{Q}|$), (F2) Edge Multiplicity (i.e., $\omega_{u,v}$), (F3) Node-level Feature (i.e., weighted degree), (F4) MHH(u, v) (see Eq. (1)), (F5) $\frac{\text{MHH}(u,v)}{\omega_{u,v}}$, (F6) Maximal Clique Indicator and (F7) Clique Cut Ratio. For each group, the classifier was retrained without the excluded features, and reconstruction accuracy was measured on both multiplicity-reduced hypergraphs and multiplicity-preserved hypergraphs. As shown in Fig. 4, F2 (Edge Multiplicity), F1 (Clique Size), and F4 (MHH(u, v)) consistently emerge as the most influential features, contributing significantly to reconstruction accuracy. In contrast, F7 (Clique Cut Ratio), F6 (Maximal Clique Indicator), and F5 ($\frac{\text{MHH}(u,v)}{\omega_{u,v}}$) show lower importance, indicating a lesser impact on reconstruction performance.

J. Additional Related Works

Community Detection: Community detection aims to identify densely connected groups of nodes within a graph, often relying on measures like edge density or modularity to define communities. Some community-detection methods [4], [5] allow overlapping communities, meaning a node may belong to multiple communities, analogous to nodes participating in multiple higher-order interactions.

However, community detection fundamentally differs from hypergraph reconstruction. Communities are defined by relative edge density and can include incomplete subgraphs, whereas higher-order interactions, each of which is an indivisible structure, are always represented as cliques. Moreover, while hyperedges commonly occur multiple times, repeated instances of the same community have rarely been considered. Due to these reasons, community detection methods are inapplicable or, at best, suboptimal for hypergraph reconstruction.

Clique Decomposition: Clique decomposition methods serve as a useful substep for identifying higher-order interactions in network data by breaking down the graph into fully connected subgraphs, or cliques. These methods are relatively straightforward to implement, supported by well-established algorithms for exact clique enumeration [2], [14] and sampling [15]. However, clique decomposition alone is insufficient to fully capture true higher-order interactions due to the flexibility in decomposition choices. For instance, a triangle can be represented as a single 3-clique or as three 2-cliques, leading to multiple valid decompositions.

To address this ambiguity, Clique Covering methods extend the idea of decomposition by identifying the smallest set of cliques that covers all edges in a graph. Thus, we included Clique Covering [6] as one of our baselines to evaluate its effectiveness in hypergraph reconstruction.

I. Appendix: Proofs

A. Proof of Lemma 1

Proof. In the projected graph $\mathcal{G}$, the edge weight w_{uv} represents the total number of hyperedges in the original hypergraph $\mathcal{H}$ that include both u and v.

For higher-order hyperedges containing u and v, each such hyperedge must include at least one additional node z, where $z \in N(u) \cap N(v)$. The maximum number of hyperedges involving u, v, and z is limited by $\min(w_{uz}, w_{vz})$, since w_{uz} and w_{vz} represent the co-occurrence counts of u, z and v, z, respectively.

Summing over all common neighbors z, the maximum contribution of higher-order hyperedges to w_{uv} is:

$$\text{MHH}(u, v) = \sum_{z \in N(u) \cap N(v)} \min(w_{uz}, w_{vz}).$$

This provides an upper bound on the number of higher-order hyperedges contributing to w_{uv}, which also includes direct (size-2) hyperedges. Thus, MHH(u, v) bounds the portion of w_{uv} attributable to higher-order hyperedges. □

B. Proof of Lemma 2

Proof. The total edge weight $w_{u,v}$ represents the total number of hyperedges in $\mathcal{H}$ that include both u and v, counting multiplicities. This total comprises contributions from both higher-order hyperedges and direct size-2 hyperedges between u and v.

From Lemma 1, the maximum possible contribution to $w_{u,v}$ from higher-order hyperedges is $\text{MHH}(u, v)$. Therefore, the minimum number of direct size-2 hyperedges is at least:

$$\text{Number of size-2 hyperedges} \geq w_{u,v} - \text{MHH}(u, v) = r_{u,v}.$$

Thus, $r_{u,v}$ serves as a lower bound on the number of true hyperedges in $\mathcal{H}$ that consist solely of $\{u, v\}$. □

C. Proof of Lemma 3

Proof. In the `Filtering` step, for each edge $(u, v) \in \mathcal{E}_{\mathcal{G}}$, we compute the sum of minimum weights with common neighbors. Finding the common neighbors of u and v takes $O(|N(u)|+|N(v)|)$ time, where $|N(u)|$ is the degree of node u. Since $|N(u)| \leq d_{\max}$ for all $u \in V$, the time to process one edge is $O(d_{\max})$. Therefore, the total time complexity is $O(|\mathcal{E}_{\mathcal{G}}| \cdot d_{\max})$. □

D. Proof of Lemma 4

Proof. The `BidirectionalSearch` step processes each clique $C \in \mathcal{C}$ up to $m_C = \min_{(u,v)\in C} \omega_{u,v}$ times. After m_C processings, at least one edge in C has its multiplicity reduced to zero, causing C to cease being a clique in the graph. Each processing involves four operations: feature computation, classification, existence checking, and graph updates.

Feature computation considers all node pairs in C, resulting in a time complexity of $O(|C|^2)$. Classification is handled by a fixed-size MLP with constant input dimension F and hidden layer size h, making the classification step $O(1)$ per clique. Existence checking and graph updates also require $O(|C|^2)$, as they iterate over all edges in C.

When the maximum clique size is bounded by $K = \max_{C\in\mathcal{C}} |C|$, the time complexity for feature computation, existence checking, and graph updates becomes $O(K^2)$. If K is small (e.g., $K \leq 88$), the per-processing cost effectively reduces to $O(1)$, as K^2 becomes independent of the graph size or the total number of cliques.

The total number of clique processings is $N = \sum_{C\in\mathcal{C}} m_C$. Across all iterations, sorting cliques by their predicted scores incurs a cumulative time complexity of $O(N \log |\mathcal{C}|)$, where $|\mathcal{C}|$ is the total number of cliques. As sorting dominates when $\log |\mathcal{C}|$ is significant, the overall time complexity of the `BidirectionalSearch` step is $O(N \log |\mathcal{C}|)$.

□

REFERENCES

[1] Y. Wang and J. Kleinberg, "From graphs to hypergraphs: Hypergraph projection and its reconstruction," in *ICLR*, 2024.

[2] E. Tomita, A. Tanaka, and H. Takahashi, "The worst-case time complexity for generating all maximal cliques and computational experiments," *Theoretical computer science*, vol. 363, no. 1, pp. 28–42, 2006.

[3] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *KDD*, 2019.

[4] G. Palla, I. Derényi, I. Farkas, and T. Vicsek, "Uncovering the overlapping community structure of complex networks in nature and society," *nature*, vol. 435, no. 7043, pp. 814–818, 2005.

[5] M. Coscia, G. Rossetti, F. Giannotti, and D. Pedreschi, "Demon: a local-first discovery method for overlapping communities," in *KDD*, 2012.

[6] A. Conte, R. Grossi, and A. Marino, "Clique covering of large real-world networks," in *SAC*, 2016.

[7] J.-G. Young, G. Petri, and T. P. Peixoto, "Hypergraph reconstruction from network data," *Communications Physics*, vol. 4, no. 1, p. 135, 2021.

[8] M. T. Do, S.-e. Yoon, B. Hooi, and K. Shin, "Structural patterns and generative models of real-world hypergraphs," in *KDD*, 2020.

[9] Y. Kook, J. Ko, and K. Shin, "Evolution of real-world hypergraphs: Patterns and models without oracles," in *ICDM*, 2020.

[10] G. Lee, M. Choe, and K. Shin, "How do hyperedges overlap in real-world hypergraphs?-patterns, measures, and generators," in *WWW*, 2021.

[11] S. Hu, X. Wu, and T. H. Chan, "Maintaining densest subsets efficiently in evolving hypergraphs," in *CIKM*, 2017.

[12] A. R. Benson, R. Abebe, M. T. Schaub, A. Jadbabaie, and J. Kleinberg, "Simplicial closure and higher-order link prediction," *Proceedings of the National Academy of Sciences*, vol. 115, no. 48, pp. E11 221–E11 230, 2018.

[13] M. E. Wall, A. Rechtsteiner, and L. M. Rocha, "Singular value decomposition and principal component analysis," in *A practical approach to microarray data analysis*. Springer, 2003, pp. 91–109.

[14] C. Bron and J. Kerbosch, "Algorithm 457: finding all cliques of an undirected graph," *Communications of the ACM*, vol. 16, no. 9, pp. 575–577, 1973.

[15] S. Jain and C. Seshadhri, "A fast and provable method for estimating clique counts using turán's theorem," in *WWW*, 2017.